EECE2310 Introduction to Digital Design and Computer Architecture
Summer 2 2025 Semester

Homework 4

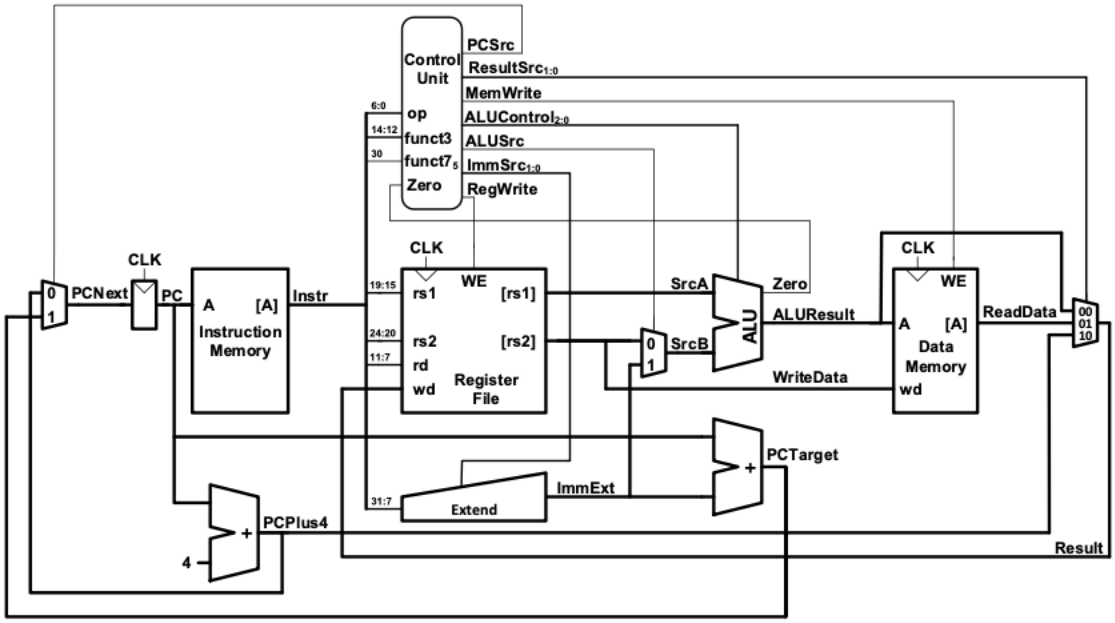
EECE2310 Course Section 01

Name: Hang Hang

[redacted]

Date Submitted:
8/6/2025

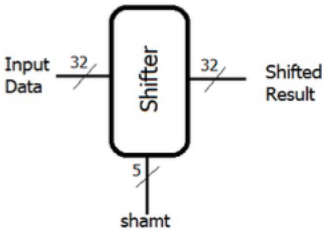
Q1 (25 Points)



Extend the above single-cycle datapath and control block diagram by adding the shifter shown below to implement the **`slli`** instruction. This shifter must be added to the datapath as a separate circuit from the ALU. You also must add a new control signal named **`Shift`**, as we do not want to change the number of inputs to the result source mux. The **`slli`** instruction has the following format (which is similar to the I-type format) **`slli rd,rs1,imm`**

31:20	19:15	14:12	11:7	6:0
[31:25]=0 [24:20] = imm	rs1	0x1	rd	0010011
12 bits	5 bits	3 bits	5 bits	7 bits

The instruction shifts the value in the **`rs1`** left number of bits equal to the 5-bit value stored in the **`imm`** field and stores that value in register **`rd`**. The figure shows the shifter to be used.



a) Draw the extended block diagram to include the new proposed control signal, the shifter circuit, and any additional circuits needed. Make sure to label any added line following the same labeling technique shown in the above block diagram.

b) Extend the "*Control Unit - Instruction Decoder*" table in the lecture slides to reflect the support for the new **`slli`** instruction. Add to the table a new row for the **`slli`** instruction and a new column for the new **`Shift`** signal.

(a)

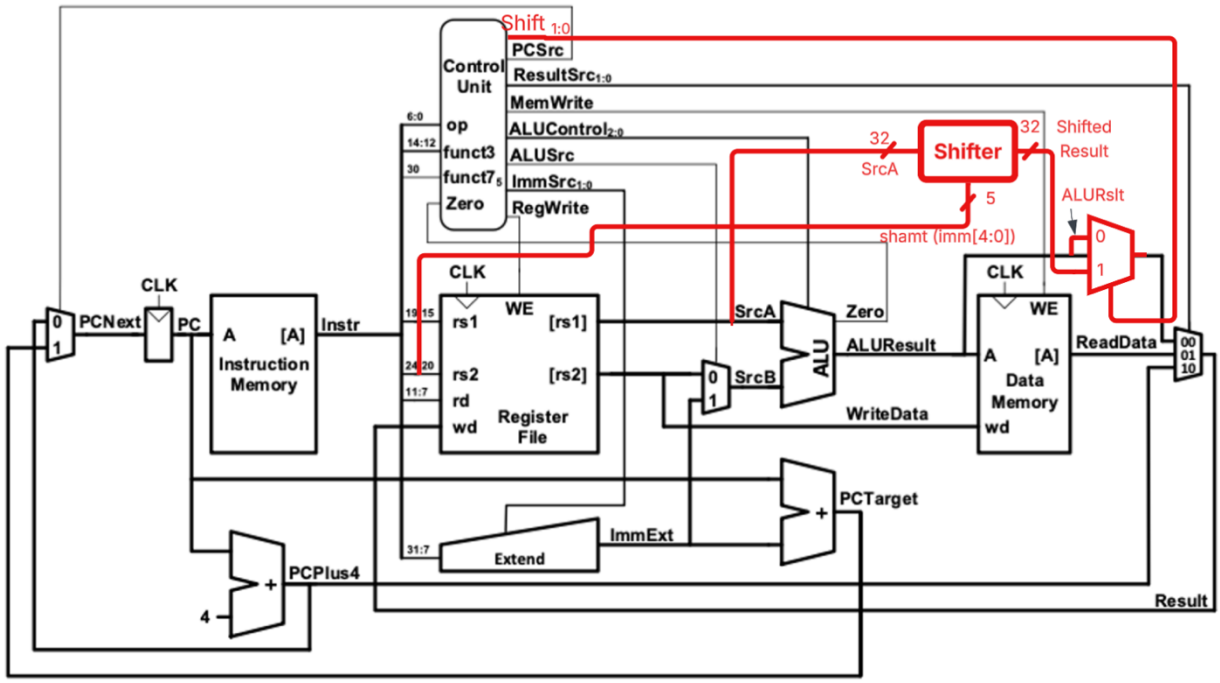


Figure 1: Single cycle datapath with shifter added.

Changes for adding the shifter:

- 5 wires and a 2-to-1 mux for selecting between ALU and shifter outputs. (Could be 4 wires, but Lucid's fixed mux input spacing requires an extra routing wire for ALUResult.)
- Register File (rs1) → [Shifter Input=SrcA]
- ImmExt (imm[4:0]) → [Shifter Shift Amount or shamt]
- The Shift signal acts as the select line for this mux:
- Shift = 0: ALU output passes through.
- Shift = 1: Shifter output passes through.

(b) Control Unit – Instr Decoder

op	Instruct.	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUControl	Jump	Shift
3	lw	1	00	1	0	01	0	000	0	0
35	sw	0	01	1	1	XX	0	000	0	0
51	R-type	1	XX	0	0	00	0	Depends on funct	0	1
99	beq	0	10	0	0	XX	1	001	0	0
19	I-type	1	00	1	0	00	0	Depends on funct	0	1
111	jal	1	11	X	0	10	0	XXX	1	0
19	slli	1	00	X	0	10	0	XXX	0	1

Q2 (25 Points)

Assume the elements in the RISC-V single-cycle datapath and control have the following delays in picoseconds (ignore the delay time through the Control Unit).

Element	Parameter	Delay (ps)
PC clock-to-Q	t_{pcq_PC}	30
Multiplexer	t_{mux}	25
ALU	t_{ALU}	200
Data and Instruction Memory (read or write)	t_{mem}	250
Register file (read or write)	t_{RF}	150
Plus 4 Adder	t_{add4}	70
Extend unit	t_{ext}	15

a) For the following RISC-V add instruction, calculate the total time in *ps* needed from the moment the clock edge triggers the PC to read the instruction address until the sum data is ready to be written to the register file right before the next clock edge.

```
add t1, t2, t3
```

b) Repeat part (a) for the following store instruction. Here you need to calculate the total time in *ps* needed from the moment the clock edge triggers the PC to read the instruction address until the destination address, of the data to be stored, is ready to be provided to the data memory right before the next clock edge.

```
sw t0, 32(s3)
```

Show all the steps of your calculations. Note: This is not the T_c as in the lecture and hence no need to consider the setup time of the register file or the data memory.

$$T_c = t_{pcq_PC} + t_{mem} + \max(t_{RFread}, t_{mux}) + t_{ALU} + t_{mux}$$

(a) add t1, t2, t3

Critical path: PC → I-Mem → RF Read → ALUSrc Mux → ALU → ResultSrc Mux

	Component	Operation	Delay (ps)	Total (ps)
1	PC Register (t_{pcq_PC})	Clock-to-Q output	30	30
2	I-Mem (t_{mem})	Fetch ADD instruction	250	280
3	Register File (t_{RFread})	Read t2 and t3 values	150	430
4	ALUSrc Mux (t_{mux})	Select register data for ALU input	25	455
5	ALU (t_{ALU})	Perform addition (t2 + t3)	200	655

6	ResultSrc Mux (t_{mux})	Route ALU output to register write	25	**680**

Thereforce, T_c or total delay=680ps until the sum is ready at the Register File write-data input.

(b) sw t0, 32(s3)

Critical path: PC → I-Mem → RF Read → ALU

	Component	Operation	Delay (ps)	Total (ps)
1	PC Register (t_{pcq_PC})	Clock-to-Q output	30	30
2	I-Mem (t_{mem})	Read the sw instruction	250	280
3	Register File (t_{RFread})	read rs1 (s3), rs2 (t0) values	150	430
5	ALU (t_{ALU})	Perform the operation: address=base+offset=s3+32	200	**630**

The result is 630ps until the store's address and data are both available to memory.

Q3. (25 Points)

For a system that uses the **little-endian** data byte ordering and right after executing the following RISC-V instructions, what will be the 32-bit contents of registers **t1**, **t2**, **t3**, **t4**, **t5** (in **hexadecimal presented as 0xAABBCCDDEE**)? You must show all the steps you used to get your answer and explain your results on a layout that shows the data memory contents for every addressable byte in the memory. Start the data segment at address zero and apply **automatic alignment** on its contents.

```
.data
    AH: .half 0x1234, 0x8899, 0x5555
    AW: .word 0x66667777, 0x00ABCDEF
.text
 main:
    la   t0, AH
    lw   t1, 0(t0)
    lb   t2, 1(t0)
    lb   t3, 2(t0)
    lbu  t4, 3(t0)
    lw   t5, 8(t0)
     ...
```

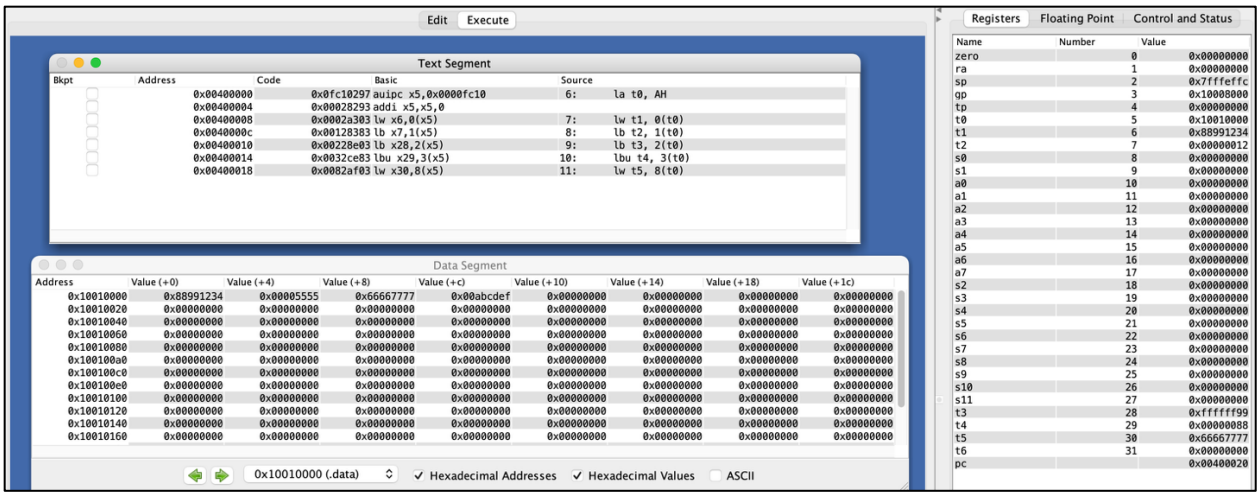


Figure 1: Results after executing the given RISC-V instructions.

<u>Explanation of each instruction within Figure 1</u>:

Address starts at 0x10010000 (the following are shortened to last 4 digits)

Address: 0x0000 0x0001 0x0002 0x0003 0x0004 0x0005 0x0006 0x0007 0x0008 0x0009
Bytes: 34 12 99 88 55 55 00 00 77 77
└──────lw t1, 0(t0)──────┘ └not accessed┘ └─padding──┘ └─lw t5, 8(t0)
│ │ │
lb t2, 1 (t0) lbu t4, 3(t0)
lb t3, 2(t0)
Address: 0x000A 0x000B 0x000C 0x000D 0x000E 0x000F
Bytes: 66 66 EF CD AB 00
└──────┘ └──────not accessed──────┘

Instruction	Address used (base+offset)	Bytes read (little-endian: LSB first)	Final register result (final 32-bit val)
`la t0, AH`	0x10010000	-	t0=0x10010000
`lw t1, 0(t0)`	0x10010000 = 0x10010000 + 0	34 12 99 88 --> 0x88991234	t1=0x88991234
`lb t2, 1(t0)`	0x10010001 = 0x10010000 + 1	12 --> sign-ext. (00010010_2, MSB=0) --> 0x00000012	t2=0x00000012
`lb t3, 2(t0)`	0x10010002 = 0x10010000 + 2	99 --> sign-ext. (10011001_2, MSB=1)--> 0xFFFFFF99	t3=0xFFFFFF99
`lbu t4, 3(t0)`	0x10010003 = 0x10010000 + 3	88 --> zero-ext. --> 0x00000088	t4=0x00000088
`lw t5, 8(t0)`	0x10010008 = 0x10010000 + 8	77 77 66 66 --> 0x66667777	t5=0x66667777

*Little-endian only matters for lw.

Rule of auto data alignment (Lecture 11, slide 20):

- .half (halfword – 2 bytes) needs to be stored at an even-numbered address (i.e., 0x0, 0x2, 0x4, …)
- .word (word – 4 bytes) must start at an address divisible by 4 (i.e., 0x0, 0x4, 0x8, …)

How these rules applied here: due to automatic alignment, the assembler inserted 2 padding bytes (0x00) between the halfword data and word data, ensuring that the first word (0x66667777) after the padding starts at address 0x0008, which is divisible by 4.

Q4 (25 Points)

Write a RISC-V program that asks the user to input string. The program then removes all space characters from the string. The program finally displays the resulting string without spaces. The following *C* code shows the proposed algorithm for the program. The string is traversed with two indices, called **`old_index`** and **`new_index`**, where the latter always takes a value less or equal to the former. When a non-space character is found, the character at position **`old_index`** is copied to position **`new_index`**, and both indices are incremented. When a space is found, the current character is not copied, and only **`old_index`** is incremented. The algorithm stops when a *null* character is found.

```c
#include <stdio.h>
int main() {
  // Read string
   char s[100];
   printf("Enter string: ");
   gets(s);
 // Remove spaces
    char c;
    int old_index = 0;
    int new_index = 0;
    do {
    // Read character
     c = s[old_index];
    // Old position moves ahead
     old_index++;
    // If it's a space, ignore
     if (c == ' ') continue;
    // Copy character
     s[new_index] = c;
    // New position moves ahead
     new_index++;
    } while (c);

  // Print result
    printf("New string: %s\n", s);
}
```

a) Write the full well-commented RISC-V program and submit it in a file named **`nospace.asm`**
b) Run your program on **RARS** and include in your homework report screen shots of the program sample runs. Make sure to enter a string containing spaces to demonstrate the correct functionality of the algorithm.

(you can use any of the RARS pseudo instructions in your program)

(a) See asm file
(b)

```
Enter string: "eece 2311 lab is awesome "
New string: "eece2311labisawesome"

-- program is finished running (0) --

Enter string: "Hello World!"
New string: "HelloWorld!"

-- program is finished running (0) --

Enter string: " "
New string: ""

-- program is finished running (0) --

Enter string: ""
New string: ""
```

Summary: Set up data section with input buffer and prompt messages. Use system calls to prompt user and read string input (reserved a max of 100 bytes). Initialize two pointers: old_index (reads original string) and new_index (writes filtered result). Loop through each character - if null terminator, exit; if space, skip; otherwise copy character to new position. After loop, add null terminator and display the space-free result.

Register	Purpose
s0	Base address of input string
s1	old_index (input index)
s2	new_index (output index)
t0	Current character being processed
t1	Temp address calculation